

Machine Learning II: K-Means Clustering for Unsupervised Learning

Week 7, Day 2: Clustering the Palmer Penguins Dataset

Victor Wandera Lumumba, MSc.

Statistician | Data Analyst | ML Expert

Mediacrest Training College

22nd Sept 2026

Abstract

This presentation introduces K-Means clustering, one of the most fundamental and widely used unsupervised machine learning algorithms.

- **Theoretical Foundation:** Understanding the mathematical intuition, assumptions, strengths, and limitations of the K-Means algorithm.
- **Data Preprocessing:** Loading, cleaning, and scaling the Palmer Penguins dataset to ensure optimal distance-based clustering.
- **Optimal Cluster Selection:** Utilizing the Elbow Method, Silhouette Score, and Davies-Bouldin Score to determine the ideal number of clusters (k).
- **Model Evaluation:** Assessing clustering performance using both internal metrics and external validation against true biological species labels.

Session Objectives

- Explain the core mechanics and assumptions of the K-Means clustering algorithm.
- Preprocess a real-world dataset (Palmer Penguins) by handling missing values and standardizing features.
- Apply quantitative methods (Elbow, Silhouette, Davies-Bouldin) to select the optimal k.
- Implement K-Means clustering using `scikit-learn` and interpret the resulting cluster centroids.
- Evaluate the unsupervised clustering results against ground-truth labels using Adjusted Rand Index (ARI) and Normalized Mutual Information (NMI).

Part 1: Theoretical Foundations

"How K-Means discovers hidden patterns."

What is Unsupervised Learning?

- **Definition:** A branch of machine learning where the algorithm learns patterns from **unlabeled data**. There is no target variable (y) to predict.
- **Goal:** To discover the underlying structure, distribution, or groupings within the data.
- **Common Applications:**
  - Customer segmentation in marketing.
  - Image compression and color quantization.
  - Document clustering and topic modeling.
  - Anomaly or outlier detection.
- **Clustering:** The task of partitioning a dataset into distinct, non-overlapping groups (clusters) where data points in the same group are more similar to each other than to those in other groups.

Introduction to K-Means Clustering

The K-Means Algorithm

K-Means is a centroid-based, iterative algorithm that partitions a dataset into k distinct, non-overlapping clusters.

- **Objective:** Minimize the Within-Cluster Sum of Squares (WCSS), also known as inertia.
- **Formula:** $WCSS = \sum_{i=1}^{k} \sum_{x \in C_i} ||x - \mu_i||^2$
- Where C_i is the i-th cluster, x is a data point in C_i, and μ_i is the centroid (mean) of C_i.
- The algorithm seeks to make clusters as **compact** and **well-separated** as possible.

How K-Means Works (The Algorithm)

1. **Initialization:** Choose the number of clusters k. Initialize k centroids randomly (or strategically using K-Means++).
2. **Assignment Step:** Assign each data point to the nearest centroid based on Euclidean distance.
3. **Update Step:** Recalculate the position of each centroid by taking the mean of all data points assigned to that cluster.
4. **Iteration:** Repeat the Assignment and Update steps iteratively.
5. **Convergence:** The algorithm stops when the centroids stabilize (i.e., assignments no longer change significantly) or a maximum number of iterations is reached.

Assumptions of K-Means

Critical Assumptions

K-Means operates optimally only when the data satisfies certain geometric properties:

- **Spherical Clusters:** The algorithm assumes clusters are spherical (isotropic) in shape. It struggles with elongated or irregular shapes.
- **Well-Separated:** Clusters should have clear boundaries with minimal overlap.
- **Roughly Equal Size:** K-Means tends to create clusters of similar variance and size. It performs poorly when true clusters have vastly different densities or sizes.
- **Numeric Data:** Requires continuous numerical features, as it relies on Euclidean distance and mean calculations.

Strengths of K-Means

- **Computational Efficiency:** Highly scalable and fast, with a time complexity of $O(n \cdot k \cdot d \cdot i)$, making it suitable for large datasets.
- **Simplicity:** Easy to understand, implement, and interpret.
- **Tight Clusters:** Guarantees convergence to a local minimum of the WCSS objective function.
- **Widespread Adoption:** Extensively supported in all major machine learning libraries (e.g., `scikit-learn`, `Spark MLlib`).

Limitations of K-Means

- **Requires Predefined k:** The number of clusters must be specified in advance, which is often unknown in real-world scenarios.
- **Sensitive to Initialization:** Random initial centroids can lead to convergence on suboptimal local minima. (Mitigated by `n_init` parameter or K-Means++).
- **Outlier Sensitivity:** The mean-based centroid calculation is heavily skewed by outliers.
- **Curse of Dimensionality:** Euclidean distance becomes less meaningful in high-dimensional spaces, requiring prior dimensionality reduction (e.g., PCA).

Part 2: The Palmer Penguins Dataset

"Exploring the biological data."

Loading Required Libraries

Python Implementation

```
import numpy as np; import pandas as pd; import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.cluster import KMeans
from sklearn.metrics import (silhouette_score, davies_bouldin_score,
                             adjusted_rand_score, normalized_mutual_info_score)
import warnings
warnings.filterwarnings('ignore'); plt.style.use('seaborn-v0_8')

```

Code Explanation

- `pandas` and `numpy`: Core libraries for data manipulation and numerical operations.
- `matplotlib` and `seaborn`: Essential for visualizing data distributions and clustering evaluation metrics.
- `sklearn.cluster.KMeans`: The primary algorithm for this session.
- `sklearn.metrics`: Provides tools to quantitatively assess cluster quality (both internally and externally).

Loading the Palmer Penguins Dataset

Python Implementation

```
# Load the dataset
df = pd.read_csv('penguins.csv')

# Display the first few rows
print(df.head())

```

Code Explanation

- `pd.read_csv()`: Reads the comma-separated values file into a pandas DataFrame.
- `df.head()`: Displays the first 5 rows to verify successful loading and get an initial glimpse of the data structure, including columns like `species`, `island`, `bill_length_mm`, and `body_mass_g`.

Dataset Overview: Shape & Features

Python Implementation

```
print("=" * 60)
print("DATASET OVERVIEW")
print("=" * 60)
print(f"\nShape: {df.shape}")
print(f"Columns: {df.columns.tolist()}")

```

Code Explanation

- `df.shape`: Returns a tuple `(344, 8)`, indicating 344 observations (penguins) and 8 distinct features.
- The features include categorical variables (`species`, `island`, `sex`), continuous numerical measurements (`bill_length_mm`, `bill_depth_mm`, `flipper_length_mm`, `body_mass_g`), and a temporal feature (`year`).

Dataset Overview: Data Types

Python Implementation

```
print("\nData Types:")
print(df.dtypes)

```

Code Explanation

- `df.dtypes`: Reveals the data type of each column.
- **Objects:** `species`, `island`, `sex` (require encoding for some algorithms, though K-Means will only use numerical features).
- **Float64:** Continuous physical measurements, crucial for distance-based clustering.
- **Int64:** The `year` column. Understanding data types is essential for proper preprocessing.

Checking for Missing Values

Python Implementation

```
print("\nMissing values per column:")
print(df.isnull().sum())

```

Code Explanation

- `df.isnull().sum()`: Counts the number of missing (`NaN`) entries in each column.
- **Findings:** The dataset contains minimal missing values. Categorical and temporal features are complete. Physical measurements have exactly 2 missing values each, and `sex` has 11 missing records.
- These minor gaps will be addressed to ensure a perfectly clean dataset for clustering.

Missing Values Percentage

Python Implementation

```
missing_pct = (df.isnull().sum() / len(df) * 100).round(2)

print("\nMissing Value Percentage:")
print(missing_pct)

```

Code Explanation

- `df.isnull().sum()` counts the missing values in each variable.
- Dividing by `len(df)` and multiplying by 100 converts the counts into percentages.
- The results show relatively low levels of missingness across the variables.
- Missing values can therefore be assessed and handled appropriately before conducting further statistical analysis.

Species Distribution

Python Implementation

```
print("\nSPECIES DISTRIBUTION")
print(df['species'].value_counts())

```

Code Explanation

- `value_counts()` calculates the frequency of each unique species.
- **Adelie:** 152 observations.
- **Gentoo:** 124 observations.
- **Chinstrap:** 68 observations.
- The distribution shows that Adelie is the most represented species, while Chinstrap is the least represented.
- These observed species labels can later be used to assess the correspondence between the unsupervised clusters and the known species.

Part 3: Data Preprocessing

"Preparing the data for distance-based algorithms."

Data Preprocessing: Dropping Missing Values

Python Implementation

```
# Create a copy to preserve raw data
df_clean = df.copy()

# Drop rows with missing values
df_clean = df_clean.dropna()
print(f"Rows after removing missing values: {len(df_clean)}")

```

Code Explanation

- `df.copy()`: Creates a working copy to prevent unintended modifications to the original raw dataset.
- `dropna()`: Removes any row containing at least one `NaN` value.
- The dataset size reduces from 344 to **333** complete records, ensuring that all subsequent distance calculations are performed on valid, non-null data.

Selecting Numerical Features

Python Implementation

```
# Define numerical features for clustering
numerical_features = ['bill_length_mm', 'bill_depth_mm',
                      'flipper_length_mm', 'body_mass_g']

# Create feature matrix X
X = df_clean[numerical_features].values
print(f"Feature matrix shape: {X.shape}")

```

Code Explanation

- K-Means relies on Euclidean distance, which only makes sense for continuous numerical data.
- We isolate the four biological measurement columns into a list.
- `.values` extracts these columns as a NumPy array (`X`), creating a clean, 2D feature matrix of shape `(333, 4)` ready for the algorithm.

Extracting & Encoding Target Variable

Python Implementation

```
# Store true species for later external evaluation
y_true = df_clean['species'].values

# Encode species to numerical values
le = LabelEncoder()
y_encoded = le.fit_transform(y_true)

```

Code Explanation

- `y_true`: Preserves the original string labels (Adelie, Gentoo, Chinstrap) for interpretable reporting.
- `LabelEncoder()`: Converts the categorical species names into integers (e.g., 0, 1, 2).
- While K-Means is unsupervised and doesn't use `y` during training, we need this encoded version to calculate external validation metrics (like ARI and NMI) after clustering.

Feature Scaling (Standardization)

Python Implementation

```
# Standardize features (CRITICAL for K-Means)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print(f"Mean after scaling: {X_scaled.mean(axis=0).round(4)}")
print(f"Std after scaling: {X_scaled.std(axis=0).round(4)}")

```

Code Explanation

- **Why Scale?** Features like `body_mass_g` (values in thousands) would completely dominate the Euclidean distance calculation over `bill_depth_mm` (values around 18).
- `StandardScaler()`: Transforms the data to have a mean of ≈ 0 and a standard deviation of 1.
- This ensures every feature contributes equally to the clustering process, preventing scale-induced bias.

Part 4: Determining Optimal k

"How many clusters are truly in the data?"

The Elbow Method (WCSS)

Python Implementation

```
inertias = []
K_range = range(2, 11)

for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_scaled)
    inertias.append(kmeans.inertia_)

```

Code Explanation

- We iterate through a range of potential cluster numbers ($k = 2$ to 10).
- For each k, we fit a K-Means model and record its `inertia_` (Within-Cluster Sum of Squares).
- **Goal:** Plot k vs. Inertia. The "elbow" point, where the rate of decrease sharply slows down, suggests the optimal k, balancing compactness and model complexity.

Silhouette Score

Python Implementation

```
silhouette_scores = []

for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = kmeans.fit_predict(X_scaled)
    score = silhouette_score(X_scaled, labels)
    silhouette_scores.append(score)

```

Code Explanation

- The Silhouette Score measures how similar a data point is to its own cluster (cohesion) compared to other clusters (separation).
- **Range:** $[-1, 1]$. A score close to 1 indicates well-defined, distinct clusters. A score near 0 indicates overlapping clusters.
- We compute this for each k and look for the k that **maximizes** this score.

Davies-Bouldin Score

Python Implementation

```
db_scores = []

for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = kmeans.fit_predict(X_scaled)
    score = davies_bouldin_score(X_scaled, labels)
    db_scores.append(score)

```

Code Explanation

- The Davies-Bouldin Index evaluates the average similarity between each cluster and its most similar cluster, based on cluster scatter and centroid distance.
- **Interpretation: Lower values are better**. A score of 0 represents ideal, perfectly separated clusters.
- We look for the k that **minimizes** this score.

Visualizing Cluster Evaluation Metrics

Python Implementation

```
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# 1. Elbow Plot
axes[0].plot(K_range, inertias, 'bo-')
axes[0].set_title('Elbow Method')

# 2. Silhouette Plot
axes[1].plot(K_range, silhouette_scores, 'go-')
axes[1].set_title('Silhouette Score')

# 3. Davies-Bouldin Plot
axes[2].plot(K_range, db_scores, 'ro-')
axes[2].set_title('Davies-Bouldin Score')

plt.tight_layout()
plt.show()

```

Optimal Cluster Recommendations

- **Silhouette Score:** Best $k = 2$. Indicates the highest level of cohesion within clusters and separation between clusters.
- **Davies-Bouldin Score:** Best $k = 2$. The lowest score reflects well-separated and compact clusters.
- **Elbow Method:** Suggests a potential point at $k = 3$, where the reduction in inertia begins to slow down significantly (diminishing returns).
- **Domain Knowledge:** We know biologically that there are exactly **3** penguin species in this dataset (Adelie, Gentoo, Chinstrap).

Key Takeaway

While mathematical metrics (Silhouette, DB) might favor $k = 2$ due to broad morphological splits, **domain knowledge** dictates that we select $k = 3$ to align with the known biological ground truth.

Part 5: Applying K-Means Clustering

"Fitting the model and interpreting the results."

Fitting the K-Means Model ($k = 3$)

Python Implementation

```
# Based on evaluation and domain knowledge, use k=3
k = 3
kmeans = KMeans(n_clusters=k, random_state=42, n_init=10, max_iter=300)

# Fit the model
kmeans.fit(X_scaled)

# Get cluster labels and add to dataframe
cluster_labels = kmeans.labels_
df_clean['cluster'] = cluster_labels

```

Code Explanation

- We initialize K-Means with $k = 3$, `random_state=42` for reproducibility, and `n_init=10` to run the algorithm 10 times with different centroid seeds to avoid local minima.
- `fit(X_scaled)`: Trains the model on the standardized data.

Within-Cluster Sum of Squares (Inertia)

Python Implementation

```
inertia = kmeans.inertia_
print(f"\nNumber of clusters: {k}")
print(f"Final Inertia (WCSS): {inertia:.2f}")

```

Code Explanation

- `kmeans.inertia_`: Returns the final WCSS value for the chosen $k = 3$ model.
- For this dataset, the inertia is approximately **370.77**.
- While this absolute number is hard to interpret in isolation, it serves as a baseline. If we were to test different preprocessing methods or feature subsets, we would aim to lower this value for the same k.

Cluster Centers (Original Scale)

Python Implementation

```
# Extract centroids and inverse transform to original scale
cluster_centers_scaled = kmeans.cluster_centers_
cluster_centers = scaler.inverse_transform(cluster_centers_scaled)

print(pd.DataFrame(cluster_centers, columns=numerical_features))

```

Code Explanation

- `kmeans.cluster_centers_`: Provides the centroid coordinates in the *standardized* space (mean=0, std=1).
- `scaler.inverse_transform()`: Crucial step that converts these centroids back to the original biological units (millimeters and grams).
- This allows us to interpret the clusters meaningfully (e.g., "Cluster 1 represents penguins with the highest average body mass and flipper length").

Cluster Distribution

Python Implementation

```
print("\nCluster distribution:")
print(df_clean['cluster'].value_counts().sort_index())

```

Code Explanation

- `value_counts().sort_index()`: Shows how the 333 penguins are distributed across the 3 predicted clusters.
- **Results:** Cluster 0 has 129 penguins, Cluster 1 has 119, and Cluster 2 has 85.
- The uneven distribution reflects natural variation in the dataset. Despite the imbalance, all clusters are substantial enough to represent distinct morphological groups, making them suitable for comparative analysis against true species.

Part 6: Model Evaluation

"Validating the unsupervised discoveries."

Internal vs. External Evaluation Metrics

Metric	Type	Description	Interpretation
Silhouette Score	Internal	Measures cohesion vs. separation.	Closer to 1 is better.
Davies-Bouldin	Internal	Avg. similarity between clusters.	Closer to 0 is better.
Calinski-Harabasz	Internal	Ratio of between/within dispersion.	Higher is better.
Adjusted Rand Index (ARI)	External	Similarity to true labels, adjusted for chance.	1 = perfect, 0 = random.
Norm. Mutual Info (NMI)	External	Shared information between clusters and labels.	1 = perfect match.

Note

Internal metrics evaluate the data structure alone. External metrics require ground-truth labels (available here for validation purposes).

Evaluating K-Means Performance

Python Implementation

```
# Internal metrics
sil = silhouette_score(X_scaled, cluster_labels)
db = davies_bouldin_score(X_scaled, cluster_labels)
# External metrics (vs true species)
ari = adjusted_rand_score(y_encoded, cluster_labels)
nmi = normalized_mutual_info_score(y_encoded, cluster_labels)
print(f"Silhouette: {sil:.4f} | Davies-Bouldin: {db:.4f}")
print(f"ARI: {ari:.4f} | NMI: {nmi:.4f}")

```

Code Explanation

- **Internal:** Silhouette (0.4462) indicates moderate separation. Davies-Bouldin (0.9420) is acceptable.
- **External:** ARI (0.7994) and NMI (0.7899) are both high (close to 1).
- **Conclusion:** Despite being an unsupervised method, K-Means successfully identified groupings that strongly align with the actual biological penguin species.

Comparing Cluster Predictions vs. True Species

Python Implementation

```
# Create comparison table
comparison = pd.DataFrame({
    'True Species': y_true, 'Cluster Label': cluster_labels})
# Generate cross-tabulation
print(pd.crosstab(comparison['True Species'],
                  comparison['Cluster Label'],
                  rownames=['Species'], colnames=['Cluster']))

```

Code Explanation

- `pd.crosstab()`: Creates a confusion matrix comparing unsupervised clusters to actual species.
- **Insights:**
  - **Cluster 1** perfectly captures all 119 **Gentoo** penguins.
  - **Cluster 0** mainly represents **Adelie** (124), with 22 misclassified.
  - **Cluster 2** mainly represents **Chinstrap** (63), with 5 misclassified.

Best Practices: Reusable K-Means Pipeline

Python Implementation

```
def perform_kmeans_clustering(data_path, n_clusters=3, random_state=42):
    df = pd.read_csv(data_path).dropna()
    features = ['bill_length_mm', 'bill_depth_mm', 'flipper_length_mm', 'body_mass_g']; X = df[features].values
    y_true = df['species']; scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)
    kmeans = KMeans(n_clusters=n_clusters, random_state=random_state; n_init=10)
    labels = kmeans.fit_predict(X_scaled); return {'dataframe': df, 'model': kmeans, 'scaler': scaler, 'cluster_centers': scaler.inverse_transform(kmeans.cluster_centers_),
        'metrics': {'silhouette': silhouette_score(X_scaled, labels),
                    'ari': adjusted_rand_score(y_true, labels)} }

```

Code Explanation

- Encapsulating the workflow in a function ensures reproducibility, modularity, and clean code.
- It handles loading, cleaning, scaling, modeling, and metric computation in a single, repeatable step.

Summary, Q&A, and Next Steps

- **Summary:** K-Means is a powerful, scalable unsupervised algorithm. Proper preprocessing (scaling, handling missing values) and rigorous evaluation (Silhouette, Davies-Bouldin, ARI) are critical for deriving meaningful clusters.
- **Q&A:**
  - Why did we choose $k = 3$ despite the Silhouette score favoring $k = 2$?
  - How does feature scaling impact Euclidean distance in K-Means?
- **Next Steps (Week 7, Day 2):** We will explore **Principal Component Analysis (PCA)** for dimensionality reduction and visualization of high-dimensional clusters, followed by Hierarchical Clustering and DBSCAN.

Victor Wandera Lumumba
Statistician | Data Analyst | ML Expert
Mediacrest Training College | Nairobi, Kenya
Email: lumumbavictor172@gmail.com